오픈소스 기반 데이터분석

02강

데이터 분석을 위한 파이썬 프로그래밍 1

컴퓨터과학과 정재화교수



학습목차

1. 시퀀스 슬라이싱과 컴프리헨션
2. 문자열 형식 지정
3. 컨텍스트 관리

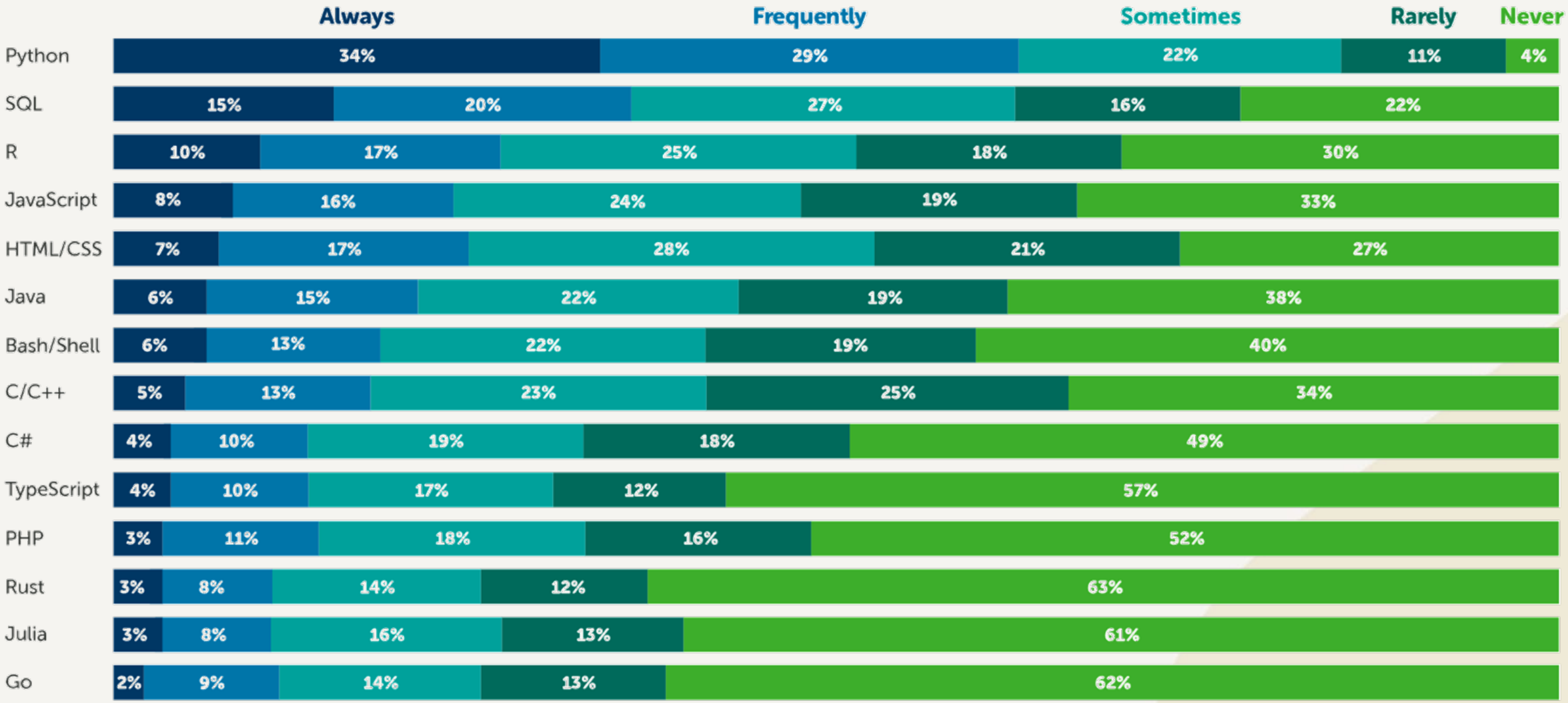


01

시퀀스 슬라이싱과 컴프리헨션



데이터 분석과 파이썬


Always
Frequently
Sometimes
Rarely
Never
Python 34% 29% 22% 11% 4%
SQL 15% 20% 27% 16% 22%
R 10% 17% 25% 18% 30%
JavaScript 8% 16% 24% 19% 33%
HTML/CSS 7% 17% 28% 21% 27%
Java 6% 15% 22% 19% 38%
Bash/Shell 6% 13% 22% 19% 40%
C/C++ 5% 13% 23% 25% 34%
C# 4% 10% 19% 18% 49%
TypeScript 4% 10% 17% 12% 57%
PHP 3% 11% 18% 16% 52%
Rust 3% 8% 14% 12% 63%
Julia 3% 8% 16% 13% 61%
Go 2% 9% 14% 13% 62%

데이터 분석과 파이썬

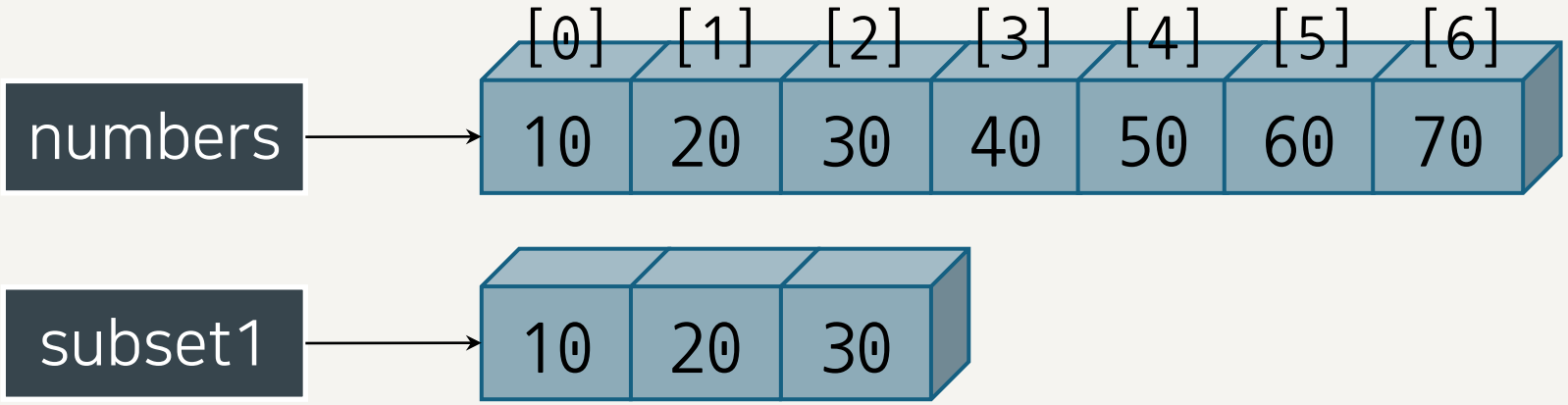
- 2021년 Anaconda Inc.의 데이터 과학자 3,104명 대상 조사에서 63%가 파이썬을 항상 또는 자주 사용
- 데이터 분석에 단순한 파이썬 프로그래밍 능력 이상이 요구
  - 데이터를 효과적으로 다루고 변환에 필요한 적절한 데이터 구조의 활용
  - 처리과정의 효율적인 표현 방법이 요구
- 파이썬 환경에서 시퀀스(리스트, 딕셔너리 등)와 같은 데이터 구조에 대한 깊은 이해
- 오류가 없는 간결한 문법적 표현으로 보다 효율적이고 가독성 높은 코드를 작성하는 방법 필요

리스트 슬라이싱 (1/3)

- 대상 리스트의 부분 리스트를 생성 또는 수정

문법형식

```
리스트이름[시작 인덱스:끝 인덱스:간격]
```

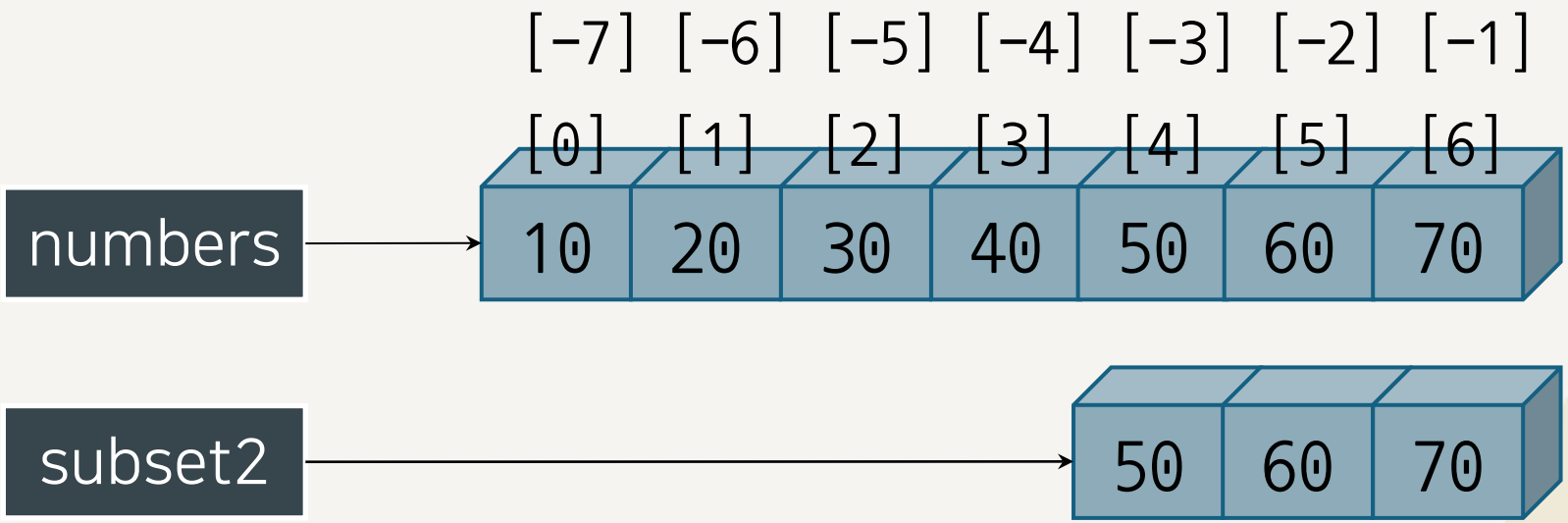



```
1 numbers = [10, 20, 30, 40, 50, 60, 70]
2 subset1 = numbers[  : 3]
3 print(subset1)
```

리스트 슬라이싱 (2/3)

리스트 슬라이싱 응용

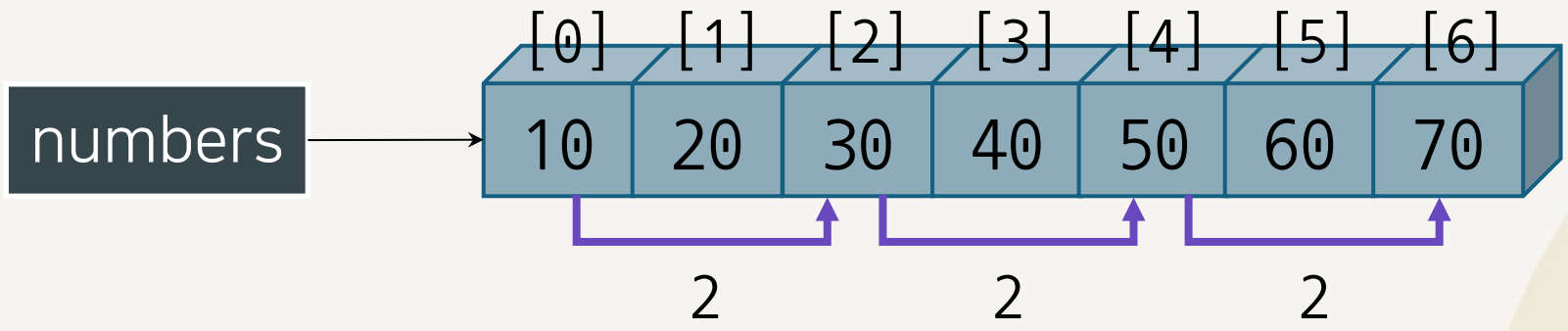
```
1 numbers = [10, 20, 30, 40, 50, 60, 70]
2 subset2 = numbers[-3 :  ]
3 print(subset2)
```



리스트 슬라이싱 (3/3)

리스트 슬라이싱 응용

```
1 numbers = [10, 20, 30, 40, 50, 60, 70]
2 list1 = numbers[  :  : 2 ]
3 print(list1)
4 list2 = numbers[  :  : -1]
5 print(list2)
```



리스트 컴프리헨션 (1/3)

리스트를 간결하게 생성·조작

문법 형식

문법형식 1

```
[표현식 for 변수 in 반복가능한 객체]
```

문법형식 2

```
[표현식 for 변수 in 반복가능한 객체 if 조건]
```

- 반복가능한 객체: 리스트, 튜플, 딕셔너리, 문자열, 집합, range 객체

리스트 컴프리헨션 (2/3)

리스트 컴프리헨션 응용

리스트 생성

```
1 numbers = []
2 for i in range(5):
3     numbers.append(i)
4 print(numbers)
```



```
1 numbers = [ i for i in range(5) ]
2 print(numbers)
```

리스트 컴프리헨션 (3/3)

리스트 컴프리헨션 응용

원소 조작

```
1 numbers = [1, 2, 3, 4, 5]
2 squares = [num ** 2 for num in numbers]
3 print(squares)
```

조건 적용

```
1 numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2 even_numbers = [num for num in numbers
                  if num % 2 == 0]
3 print(even_numbers)
```

딕셔너리 컴프리헨션 (1/2)

- 대상 딕셔너리를 간결하게 생성·조작
  - 리스트의 [ ] 연산자 대신 딕셔너리의 { } 연산자를 사용
  - 문법 형식

문법형식 1

```
{키: 표현식 for 변수 in 반복가능 객체}
```

문법형식 2

```
{키: 표현식 for 키변수, 값변수 in 딕셔너리.items()}
```

딕셔너리 컴프리헨션 (2/2)

- 딕셔너리 컴프리헨션 응용
  - 딕셔너리 생성

```
1 squares = {}
2 for i in range(5):
3     squares[i] = i ** 2
4 print(squares)
```



```
1 squares = { i: i ** 2 for i in range(5) }
2 print(squares)
```

02

문자열 형식 지정



문자열 형식 지정의 개념

- 데이터 분석 과정에서 특정 형식의 데이터를 출력하거나 변수 값을 포함한 문자열을 생성하는 상황이 자주 발생
  - 문자열 포맷팅(string formatting) 기능 사용
  - 처리과정의 효율적인 표현 방법이 요구
- 오류가 없는 간결한 문법적 표현으로 보다 효율적이고 가독성 높은 코드를 작성하는 방법 필요
  - C 스타일(% 연산자) 포맷팅
  - str.format( ) 메서드를 이용한 포맷팅
  - f-문자열 포맷팅

C 스타일(% 연산자) 포맷팅

- C 언어의 printf( ) 스타일과 유사
  - C 언어를 배운 사람이라면 쉽게 이해하고 사용
  - C와 파이썬을 함께 사용하는 환경에서 일관된 스타일 유지

문법형식

```
"형식 지정자를 포함한 문자열" % (변수 리스트)
```

- C 스타일의 단점
  - 변수의 개수가 증가할 수록 복잡해지고 가독성이 저하
  - 변수의 개수가 잘못되거나 순서 불일치 시 오류 발생
  - 딕셔너리, 리스트 등 중첩된 데이터 구조를 직접 활용하기 어려움

주요 형식 지정자

형식 지정자	설명
%d	정수(decimal) 출력
%i	정수(integer) 출력 (%d와 동일)
%o	8진수(octal) 출력
%x	16진수(lowercase) 출력
%X	16진수(uppercase) 출력
%f	소수점이 있는 실수(float) 출력
%.nf	소수점 이하 n자리까지 출력
%c	단일 문자 출력 (정수의 ASCII 문자 변환)
%s	문자열(string) 출력
%%	%문자 자체를 출력
⋮	⋮

C 스타일 포맷팅 활용

C 스타일 포맷팅 활용 예

```
1 item = "프린터"
2 price = 360000
3 print("상품명: %s, 가격: %d" % (item, price))
```

새로운 포맷팅

```
1 item = "프린터"
2 price = 360000
3 print("상품명: {}, 가격: {}".format(item, price))
```

str.format( ) 메서드를 이용한 포맷팅

- 중괄호 { }로 표시된 자리 표시자를 사용하여 문자열 내의 특정 위치에 값을 삽입하는 방식

문법형식

```
"문자열 {위치 또는 이름}".format(인자)
```

- % 연산자 방식보다 가독성이 낫고 유연
- str.format( ) 메서드의 단점
  - 복잡한 문자열 형식 지정이 필요한 경우 코드가 장황
  - 여러 줄의 복잡한 형식을 지정할 때 중괄호와 형식 지정자의 조합 시 가독성 저하

str.format( ) 메서드 활용

- 중괄호 내에는 인덱스, 키워드 또는 속성 이름을 지정
- 형식 지정자를 통해 정렬, 패딩, 소수점 자릿수, 천 단위 구분자 등 다양한 출력 형식을 지정

```
1 basic_format = '이름: {}, 나이: {}, 월급: {}원'.format(name, age,
                                                    salary)
2 index_format = '직원 {0}의 나이는 {1}세이고, {0}의 세후 월급은
                 {2}원입니다.'.format(name, age,
                                      int(salary * (1 - tax_rate)))
3 keyword_format = '직원정보: 이름: {employee} 나이: {years}세
                   월급: {income:,}원 세금: {tax:.1%} 실수령액:
                   {net_income:,}원'.format(
                   employee=name, years=age, income=salary,
                   tax=tax_rate, net_income=int(salary * (1 - tax_rate)))
```

f-문자열 (Formatted String Literal) (1/2)

- 파이썬 3.6 버전부터 도입된 문자열 포멧팅 방식
- 문자열 앞에 'f' 또는 'F' 접두사를 붙이고, 문자열 내부에 중괄호({ })를 사용하여 변수나 표현식을 표현

문법형식

```
f"문자열 {표현식}"
```

- 직관적 표현으로 코드의 길이가 줄고 가독성 향상

```
1 name = "홍길동"
2 age = 30
3 print(f"이름: {name}, 나이: {age}")
```

f-문자열 활용 예

```
1 basic_format = f'이름: {name}, 나이: {age}, 월급:
                  {salary}원'
2 index_format = f'직원 {name}의 나이는 {age}세이고,
                  {name}의 세후 월급은 {int(salary *
                  (1 - tax_rate))}원입니다.'
3 keyword_format = f'직원정보: 이름: {name} 나이:
                  {age}세 월급: {salary:,}원 세금:
                  {tax_rate:.1%} 실수령액:
                  {int(salary * (1 - tax_rate)):,}원'
```

03

컨텍스트 관리



리소스의 사용

- 리소스는 획득 및 사용 후 명시적으로 폐기
  - 리소스: 파일, 데이터베이스 연결, 네트워크 소켓 등
  - 폐기하지 않을 시 메모리 누수나 데이터 손상과 같은 문제 발생
- 자원 사용 예

```
1 file = open("output.txt", "w")
2 file.write("Hello, world!\n")
3 file.close()
```

컨텍스트 관리의 이해

- 컨텍스트 관리(context management) 정의
  - 자원을 효율적이고 안전하게 사용할 수 있도록 하는 메커니즘
  - 자원을 사용 후 명시적으로 폐기해 주어야 하는 자원의 획득과 해제를 자동으로 처리
  - with 키워드를 사용

```
1 file = open("output.txt", "w")
2 file.write("Hello, world!\n")
3 file.close()
```



```
1 with open("output.txt", "w") as file:
2     file.write("Hello, world!\n")
```

정리하기

- 리스트 슬라이싱을 통해 배열의 특정 부분을 효율적으로 추출한다.
- 리스트 컴프리헨션으로 간결한 코드 작성이 가능하다.
- 딕셔너리 컴프리헨션으로 키-값 데이터를 효율적으로 처리한다.
- C 스타일 포맷팅은 간단한 형식 지정에 유용하다.
- str.format() 메서드는 {}를 사용한 자리 표시자로 유연한 형식 지정을 제공한다.
- f-문자열은 변수를 직접 문자열에 삽입 가능한 가장 직관적인 방법이다.
- with문은 파일, 네트워크 연결 등 자원의 안전한 관리를 보장한다.
- 예외처리는 예외 발생 시에도 자원이 적절히 해제되어 코드 안정성을 향상시킨다.

03강 다음시간안내

데이터 분석을 위한 파이썬 프로그래밍 2

